

Predicting the Mean Ribosome Load of 5'UTRs Using Transformers

Marie Huynh

SUNet ID: mahuynh

Department of Biomedical Data Science

Stanford University

mahuynh@stanford.edu

Ghadi Nehme

SUNet ID: ghadi

Department of Mechanical Engineering

Stanford University

ghadi@stanford.edu

1 Introduction

The Covid Pandemic marked the emergence of messenger RNAs therapeutics in the world of medicine. The use of mRNA medicine is now expanding from covid to other diseases such as HIV, cancer, ... In the context of mRNA vaccines, once a viral protein (drug) is designed to induce immune response, we have to design an mRNA that will serve as a set of "instructions" to produce this viral protein in one's body. We are thus interested in the process of back-translation from protein to mRNA, which is illustrated on the figure 1 below.

mRNA design is hard first because of the number of possibilities. The choice of the sequence is important as Literature and experiments have shown the importance of codon choices, especially on protein expression [1]. This property is very important in optimal drug design, allowing us to design and control the level of expression of a drug in one's body. It can be "measured" using Mean Ribosome Load which is equivalent to the number of ribosomes that bind onto an mRNA. 5'UTRs, which mark the start of an mRNA (Figure 1), play a major role in protein expression. In 2019, the Seelig Lab in collaboration with Moderna Therapeutics have predicted the mean ribosome load (MRL) for a given 5'UTR sequence [2] using a Convolutional Neural Network model called Optimus 5'. In this project, we reproduce their approach (CNNs) and predict MRL using other deep learning techniques (LSTMs), classification algorithms (Multinomial Bayes) and an NLP approach (Transformers) to explore other ways to predict MRL and improve on the accuracy. The input to our algorithms is a **5' UTR sequence** which is an ordered string of nucleotides (A, T, G, C) and the output to our algorithms is the **Mean Ribosome Load (MRL)** which is a float. We manage to reach a better accuracy of $r^2 = 0.95$ with our ensemble model of Transformers on 5'UTRs of fixed length and an improved accuracy of $r^2 = 0.81$ on 5'UTRs of varying length with an LSTM model.

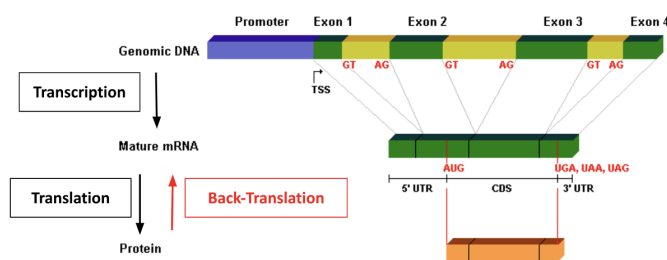


Figure 1: Central Dogma [3]

2 Related Work

Even though numerous factors have been identified as rational factors for mRNA sequence design, the latter remains incompletely understood. In the context of mRNA therapeutics, the

problem of back-translation is for now solved with algorithms looking at several characteristics of interests such as propensity to form secondary structure, stability, use of optimal codons. In most cases today, the number of possible candidates (mRNA) for a given protein are filtered in a **sequential manner with respect to one criteria at a time**. The first characteristics in the search of an optimal mRNA are often GC content, codon bias and unfavourable pairs[4].

Nevertheless, recent models such as Optimus 5' [2] or Saluki [5] published respectively in 2019 and 2022 offer promising results for designing optimal mRNA and optimizing characteristics of interests such as level of expression and half-life of a protein. Both deep learning models take as input a one-hot encoded sequence and output a wanted characteristic. On one hand, Optimus 5' [2] is a convolutional neural network (CNN) which takes as input a given 5' UTR sequence and predict its mean ribosome load. This promising tool for designing de novo 5' UTRs with controlled level of expression performs well on short 5' UTRs of length 50 with an r^2 of 0.90, but still struggles on 5'UTRs of greater length, limiting its use on the majority of humans 5'UTRs. Furthermore, the lack of data for some specific UTRs such as homopolymers or sequences with upstreams AUGs results in subsets of mispredictions. On the other hand, Saluki [5] is a hybrid convolutional and recurrent deep neural network which takes as input an mRNA sequence and predicts the mRNA half-life with pearson coefficient $r = 0.77$ on human UTRs. This model which was trained on long sequences of mRNA up to 12,288 nucleotides is considered state of the art for predicting half-life.

Finally, mRNA sequence design being a nascent field, it is only just recently that an unexpected connection with natural language has been uncovered. Transformer protein language models were first introduced by Rives, Alexander, et al. in 2019 [6], which presented a deep contextual language model trained on 86 billion amino acids across 250 million protein sequences spanning evolutionary diversity. This transformer model took as input a protein sequence and was able to predict fundamental properties of proteins such as secondary structure, contacts, and biological activity. Last October 2022, Facebook Research published the state-of-the-art ESM-2 [7] which outperformed all tested single-sequence protein language models across a range of structure prediction tasks. This transformer model takes as input a protein sequence and can be used to predict structure, function and other protein properties directly from the sequences. Keeping in mind these approaches, we will explore how to predict MRL given a 5' UTR.

3 Dataset and Features

3.1 Data and Preprocessing

We are using the data associated with the Seelig Lab study available from Gene Expression Omnibus under accession GSE114002 [8]. We considered the following two datasets:

"GSM3130435 egfp unmod 1" which contains 326 033 5'UTRs of 50 nucleotides and "GSM4084997 varying length 25 to 100" which contains 106 529 5'UTRs of 25 to 100 nucleotides. We will denote the latter **dataset 1** and **dataset 2** respectively. We denote **dataset 3** the combination of datasets 1 and 2. All datasets map a 5' UTR sequence to Mean Ribosome Load (MRL). 5' UTRs are **an ordered string of letters (ATGC)** and MRL is given under the form of **a float between 0 and 12**.

5'UTR	MRL	Total Reads
"CCACTCG...CGGCTA"	3.04	12126.0

Table 1: Example of an instance of the datasets

Each UTR is also given a measure of "total reads" (measured during the experiments to obtain the MRL). As in the paper, we got **rid of the sequences with very few reads** to ensure the best quality of the data following the thresholds indicated in the Seelig Study. Thus, we find ourselves with **280 000 5' UTRs** in Dataset 1 and **101 726 5' UTRs** in Dataset 2. For training our models, we split our data into 80%, 10% and 10% for the training, validation and test sets, respectively.

3.1.1 5' UTRs Sequences

We considered several ways to process the sequences to input them in different models. A first way is **one-hot encoding** where each letter in the string is encoded as a list of three 0's and one 1 at positions 0, 1, 2, 3 for A, C, G, T respectively. Second, we have **Tokenizing** where each letter in the string is encoded as the following : A: 1, C: 2, G: 3, T: 4. And finally, we have **k-mer counting** where we break down our 5' UTRs into overlapping "words" of length k. As an illustration, using 4-mer counting on "ATCGATCG", gives us 5 words : "ATCG", "TCGA", "CGAT", "GATC", "ATCG". We are thus considering overlapping windows of the sequence.

3.1.2 MRL

As for the MRL, we chose to consider two cases. First, we considered **Discrete Values between 0 and 12**. We tried to view the problem as a classification problem. To do so, we rounded the MRLs of each 5' UTR to the nearest integer.

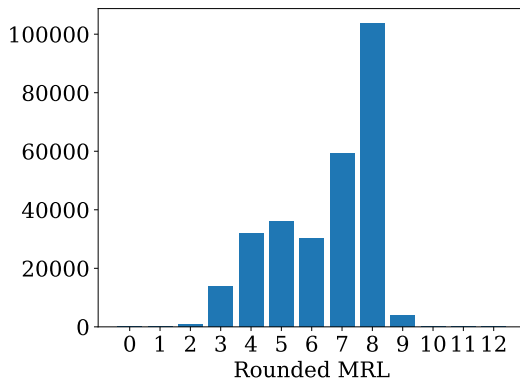


Figure 2: Class Imbalance of MRL in Dataset 1

To deal with class imbalance (Figure 2), we used different techniques such as **oversampling** and relabeling in smaller quantitative classes: Low ($MRL \leq 5$), Intermediate ($5 < MRL \leq 7$) and High ($MRL > 7$).

Finally, we also worked with **Continuous Values between 0 and 12**. MRLs are given as floats between 0 and 12. We normalize it using Standard Scaler() from sklearn to obtain better results.

3.2 Data Exploration

We started by exploring the data. One interesting observation is the distribution of the MRL over the dataset. It seems that the data is a bimodal distribution with respect to MRL. Another interesting aspect we wanted to explore was the presence of upstream ATGs (upstream start codons) which are known to influence the gene expression and thus the MRL later on. We observe that UTRs without an ATG have a higher MRL distribution. (Figure 3)

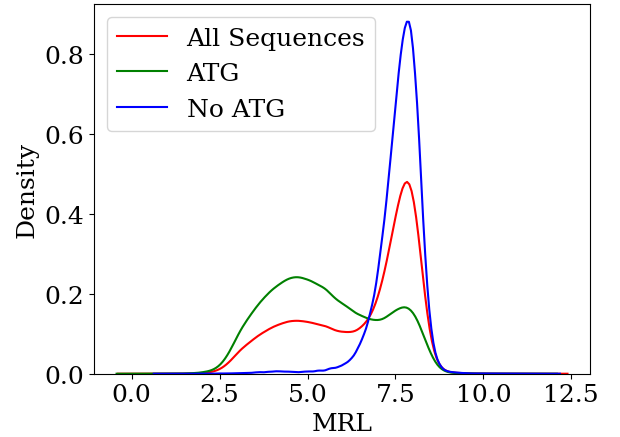


Figure 3: Distribution of MRL with or without ATG across Dataset 1.

4 Methods

In this section, we will present the different methods and models we implemented to evaluate and train our data on.

4.1 Evaluation

For the accuracy of our models which output a continuous value between 0 and 12, we will consider the **R-squared value** of the plot of the predicted MRL with respect to observed MRL:

$$r^2 = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2},$$

where $y^{(1)}, \dots, y^{(n)}$ is the observed MRLs, $\hat{y}^{(1)}, \dots, \hat{y}^{(n)}$ is the predicted MRLs and $\bar{y}$ the mean of the observed MRL. To evaluate our models which output a discrete value between 0 and 12, we will consider **accuracy**. For the loss function of our models, we used **Mean-Squared Error**:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2,$$

where $y^{(1)}, \dots, y^{(n)}$ is the observed MRLs and $\hat{y}^{(1)}, \dots, \hat{y}^{(n)}$ is the predicted MRLs.

4.2 Multinomial Naive Bayes

Multinomial Naive Bayes is a supervised learning algorithm for classification of discrete-valued features. It is often used on text classification. Based on the strong assumption that all features for a given class are independent, this algorithm is simple, easy to implement and works well on many problems.

4.3 LSTMs (Long Short-Term Memory)

Long Short Term Memory networks (LSTMs) are a type of recurrent neural networks (RNN) which address the vanishing gradient problem and are able to learn longer-term dependencies. First introduced in 1997 by Hochreiter and Schmidhuber [9], they are widely used in different applications such as speech recognition, machine translation or image classification. The idea behind LSTMs is that each computational unit is linked to a previous hidden state h as well as a state c which plays the role of memory. LSTMs will help us detect longer range patterns and thus, it has the potential of improving the results from the paper. [2]

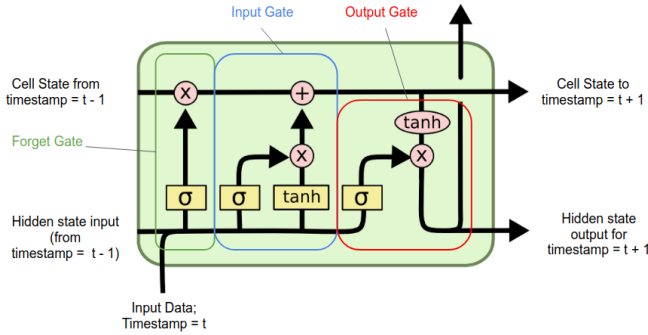


Figure 4: Scheme of an LSTM Cell [10]

An LSTM cell (Figure 4) is composed of three core parts. The **cell state** encodes the relevant information aggregated from the data from all previous time-steps that have been processed. The **hidden state** is an output of the LSTM cell, used for Prediction, containing information from previous inputs and the current input. Finally, the **gates** determine whether or not to let in a new input (input gate), remove the information (forget gate), or let it influence the output at the current time step (output gate).

4.4 CNN

Convolutional Neural Networks (CNNs) [11] are a type of neural networks often used on images. The convolutional layers apply a set of learnable filters, to the input images, which will be able to extract features from the data. In our project, we consider the one-hot encoded 5'UTRs like images. Then, by using CNNs we will be able to learn complex patterns in the sequence that could predict the MRL value of the 5'UTR.

4.5 Transformers

First introduced in the paper *Attention is all you need* [12], the Transformer model is a seq2seq model based on a self-attention mechanism, outperforming previous SOTA models on several applications. Self-attention applies attention inside a single sequence. It allows the model to analyze the interdependence of words inside a same sequence to produce a relevant representation.

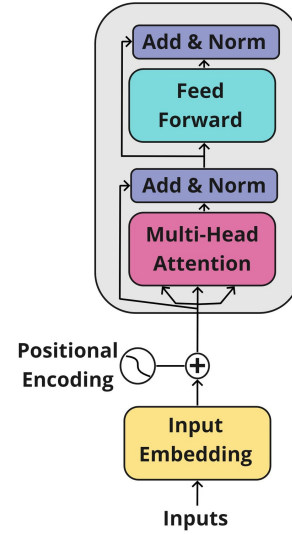


Figure 5: Scheme of a Transformer Block [13]

In our project, we use a Transformer Block (Figure 5) which consists of a positional encoding, followed by a multi-head attention layer and a feed forward neural network. Moreover, we have a residual connection and a layer normalization step between the layers of the transformer block to prevent overfitting and to ensure the stability of the algorithm. Thus, in our model, we will be only using a Transformer Encoder which will give us access to the latent space where the data is processed with self-attention. From here, we just need to pass it through some neural networks to get the MRL of the 5'UTRs.

4.6 Ensemble Model

An ensemble model is a Machine Learning technique where different models are combined to predict an outcome. The models could be different algorithms and could be trained on different datasets.

5 Experiments, Results and Discussions

In this section, we will present the different experiments we did. Then, we will show the results and analyze them.

5.1 Multinomial Naive Bayes

First, we looked at the problem from a classification point of view and wanted to try out classification models such as Multinomial Naive Bayes. We used k-mer preprocessing for the sequences, with $k = 4$, and a learning rate 0.01 for our Multinomial NB model. These hyperparameters have been obtained from a grid search in $k \in \{3, 4, 5, 6\}$ and $lr \in \{0.1, 0.01, 0.001\}$. As we have noted in section 3.1.2, our dataset is imbalanced. After trying to train this model on the initial dataset, we obtained an accuracy of 47% on the test set. After trying several approaches (oversampling, undersampling, weights, ...), we decided to relabel the MRL's as mentioned in section 3 in "low", "intermediate" and "high" and obtained an accuracy of 70% on the test set. We obtain the below "confusion matrix" (Figure 6), where we divided each cell by the total amount of actual values observed for the given class.

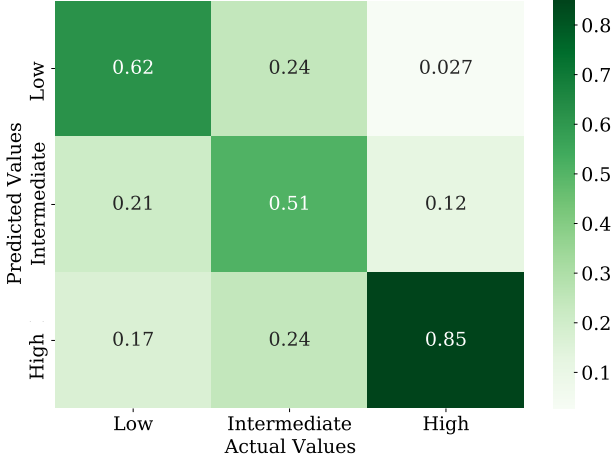


Figure 6: Confusion Matrix of Multinomial NB on 5'UTRs with fixed length, accuracy: 0.70

The lowest accuracy is obtained for the intermediate values (51 % accuracy) which is the hardest category to learn for the model. Mispredictions for the intermediate class are equally shared between high and low. Furthermore, the low class is less well learned by the model than high class due to not enough data for low MRLs.

5.2 CNNs and LSTMs

To improve on accuracy, we started by reproducing the results of [2] with the CNN model, Optimus 5'. We took the one-hot encoded sequence and predicted the MRL as a float. Optimus 5' [2] performs with $r^2 = 0.90$ on the testing set of Dataset 1 (Figure 7) and $r^2 = 0.73$ on the Dataset 2.

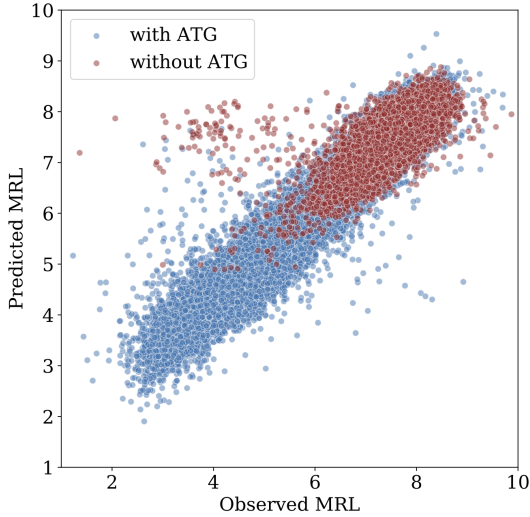


Figure 7: Predicted MRL vs. Observed MRL, for Dataset 1, using Optimus 5', R-squared: 0.90 (Model of the paper)

During our data exploration, we saw that MRLs without an upstream ATG have a higher MRL (for biological reasons mentioned in [2]) and the model was able to learn that pattern observed in data. Nevertheless, on Figure 7, we observe a set of mispredictions for sequences with no ATG which are overpredicted, because of lack of examples of sequences with no ATG and a low MRL.

To compare, we built an RNN with 3 LSTM layers and 2 Dense Layers. We observed an improvement on Dataset 1 with

$r^2 = 0.94$ on the test set (Figure 8), but a less good performance on the Dataset 2 (varying length) with $r^2 = 0.63$. We did a grid search of the number of layers and took the model with best validation r-squared.

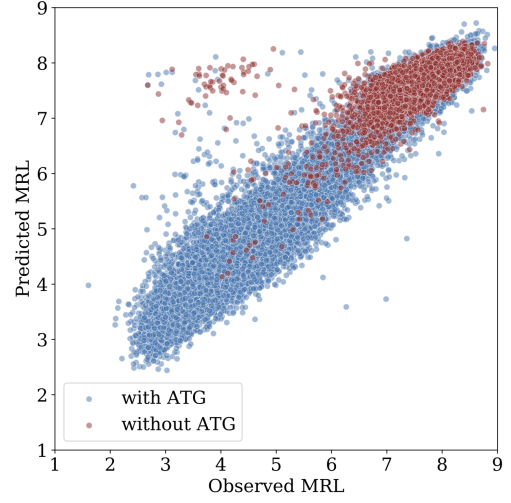


Figure 8: Predicted MRL vs. Observed MRL, for Dataset 1, using an RNN: 3 LSTM layers, R-squared: 0.94.

On Figure 8, we observe that we obtain similar results with an LSTM model than for the CNN Model, with an increase in accuracy. We observe a similar phenomenon as with Optimus 5': there is a subset of UTRs without an upstream ATG that are overpredicted by more than a factor of two. Nevertheless, this phenomenon seems to be slightly less present in our LSTM model than in Optimus 5'.

To increase the performance of our LSTM model on varying length, we decided to combine dataset 1 and 2 since we saw dataset 2 (varying length) had fewer 5'UTRs for each different lengths. After training on the resulting dataset 3, we managed to get $r^2 = 0.85$ on the test set of dataset 3. By doing that, we gave the model a bigger dataset which allowed a better observation of patterns, and thus, a better prediction for 5'UTRs of varying lengths as can be seen in table 2.

Training Set	Model	Test Set	r^2
Dataset 2	Optimus 5'	Dataset 2	0.73
Dataset 2	LSTM	Dataset 2	0.63
Dataset 3	LSTM	Dataset 2	0.81

Table 2: R^2 comparison of Different Models on Dataset 2

5.3 Transformers

5.3.1 Transformer Blocks

Given the capacity of Transformers to make accurate predictions based on a sequence of words and its context, we decided to use Transformers on our data.

We started by splitting our 5'UTRs into k-mers, overlapping windows of k letters, which will be our input. In order to choose the value of "k", we trained the model for values of $k \in \{1, 2, \dots, 9\}$ and computed the r-squared value over the validation set as can be seen on Figure 9. We observe a plateau between $k = 3$ and $k = 6$, so, we decided to pick $k = 5$ for our k-mer paramater. We thus split 5'UTRs in 5-mers, tokenize them and train our model using the mean squared error loss. We test our model. We obtain $r^2 = 0.92$, with same pattern of mispredictions in the upper left of Figure 10.

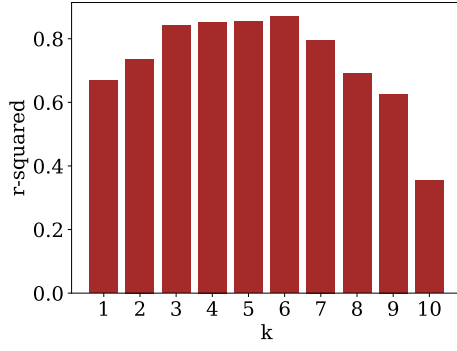


Figure 9: R-squared computed over the validation set, for models trained using k-mers, as a function of k.

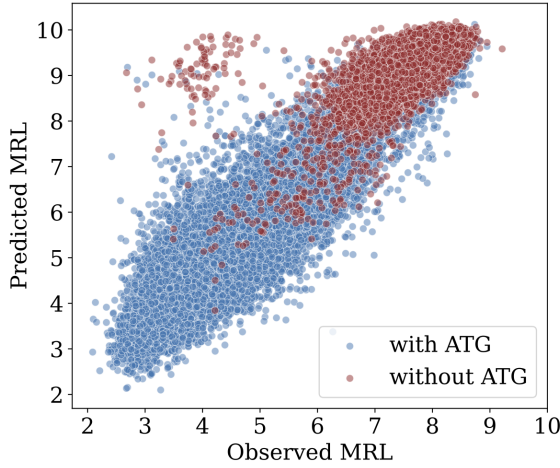


Figure 10: Predicted MRL vs. Observed MRL, for Dataset 1, using Transformers, $r^2 = 0.92$.

We observe that transformers manage to get an r-squared of 0.92. Nonetheless, as we can see on Figure 10, this model similarly to LSTM, overestimates 5'UTRs without ATG which can be observed in the upper left corner.

5.3.2 Ensemble model with Transformers

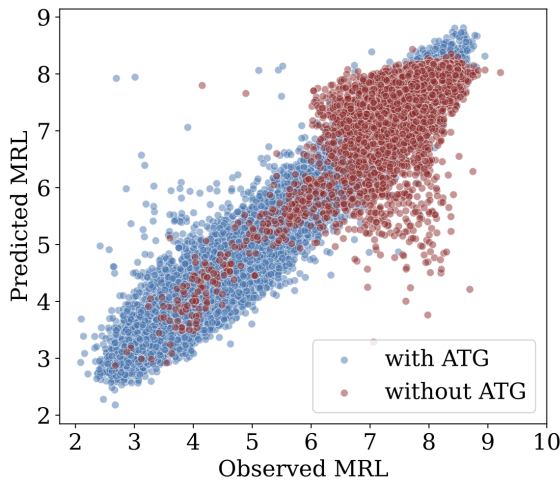


Figure 11: Predicted MRL vs. Observed MRL, for Dataset 1, using the ensemble model described after, $r^2 = 0.95$.

Given this issue, we tried to train a model for 5'UTRs with ATG and another model for 5'UTRs without ATG and combine them into an ensemble model. Moreover, given that the distribution of the MRL values of the 5'UTRs without ATG tends to be high, we decide to oversample the 5'UTRs with low MRL, in order to balance the data.

Therefore, we train one model consisting of a transformer block followed by an LSTM layer on the oversampled data of 5'UTRs without ATG and another one on the data of 5'UTRs with ATG. We combine them using an If statement: "contains ATG?" and reached on accuracy of $r^2 = 0.95$ on the test set on dataset 1, our best performance yet. As we can see on Figure 11, this ensemble model combined with oversampling the low-MRL solved the overpredictions of MRL values for the 5'UTRs without ATG, with some limitations : mispredictions of high MRL predicted as intermediate MRL ($\text{MRL} \in [5, 7]$).

6 Conclusion

In conclusion, we explored diverse machine learning algorithms to predict the Mean Ribosome Load of 5'UTRs and improved on the accuracy of the CNN model Optimus 5' [2]. We started out with a baseline model where we used Multinomial Naive Bayes to predict whether the MRL corresponding to a 5'UTR was low, intermediate or high which reached 70% accuracy using Dataset 1. We then reproduced the Optimus 5' which performed with a high accuracy on fixed length ($r^2 = 0.90$) but was less performant on varying lengths ($r^2 = 0.73$). Several limitations appeared in the datasets associated with the study and the model: the class imbalance from a classification point of view or the bimodal distribution from a continuous point of view, the lack of data for some specific types of UTRs (UTRs with no ATGs and low MRL, homopolymer UTRs, ...), and the model's limitations on varying length. Hypothesizing that we could use LSTM's learning of longer range dependencies, we built and trained an LSTM model. Despite again some overpredictions of a subset of sequences with no ATG, the latter phenomenon was less present than predictions with Optimus 5' and we obtained an improved accuracy on fixed length ($r^2 = 0.94$). To improve on varying length, we combined datasets 1 and 2 to train our LSTM model and actually obtained an improved accuracy of $r^2 = 0.81$ on the testing set of dataset 2. To improve on our set of mispredictions on fixed length and the bias of the model concerning the presence of upstreams ATGs, we built an ensemble model which combines two neural networks with transformers trained on 5'UTRs with ATG and with no ATG respectively using dataset 1. This model—combined with oversampling—allowed us to solve the overprediction of MRLs of 5'UTRs with no ATG and we obtained a final accuracy ($r^2 = 0.95$) on dataset 1, our best performance.

For future works, it would be interesting to implement a transformer model or an ensemble model similar to the one stated above on dataset 2 (varying length 5'UTRs) and do more analysis on the attention computed to see what motifs are used to predict. It would also be interesting to incorporate in our analysis more biological motifs like Kozak sequences which are known to have an important role in 5' UTRs (too few examples in our datasets) and improve performance of our model by looking for more diverse data of UTRs. Finally, 3' UTRs' role remains uncompletely understood and a similar analysis could be conducted on them to predict gene expression or other characteristics of interest.

7 Contributions

We met every week to work together on the project. We explored the data together, discussed what experiments we were planning on doing, analyzed the outputs together and set our future directions. We contributed equally to each part of the project.

References

- [1] Tessa EF Quax, Nico J Claassens, Dieter Söll, and John van der Oost. Codon bias as a means to fine-tune gene expression. *Molecular cell*, 59(2):149–161, 2015.
- [2] Paul J Sample, Ban Wang, David W Reid, Vlad Presnyak, Iain J McFadyen, David R Morris, and Georg Seelig. Human 5 utr design and variant effect prediction from a massively parallel translation assay. *Nature biotechnology*, 37(7):803–809, 2019.
- [3] Lou Heaton. Making sense out of the visual representation of transcription, 2022.
- [4] Claes Gustafsson, Sridhar Govindarajan, and Jeremy Minshull. Codon bias and heterologous protein expression. *Trends in biotechnology*, 22(7):346–353, 2004.
- [5] Vikram Agarwal and David R Kelley. The genetic and biochemical determinants of mrna degradation rates in mammals. *bioRxiv*, 2022.
- [6] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry Ma, et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021.
- [7] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic level protein structure with a language model. *bioRxiv*, 2022.
- [8] Seelig G Sample PJ, Wang B. Human 5 utr design and variant effect prediction from a massively parallel translation assay, <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE114002>.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [10] Ryan T. J. J. Lstms explained: A complete, technically accurate, conceptual guide with keras, 2020.
- [11] Keiron O’Shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [13] Serena McDonnell, Omar Nada, Muhammad Rizwan Abid, and Ehsan Amjadian. Cyberbert: A deep dynamic-state session-based recommender system for cyber threat recognition. In *2021 IEEE Aerospace Conference (50100)*, pages 1–12. IEEE, 2021.
- [14] Francois Chollet et al. Keras, 2015.
- [15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [16] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pages 265–283, 2016.
- [17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.
- [18] Quadeer Shaikh. Transformers for text classification, 2022.

All code has been written using the following libraries and sources:

- Keras [14]
- Scikit-Learn [15]
- Tensorflow [16]
- Scipy [17]
- Transformers for Text Classification [18]